Propuesta de Proyecto de Seguridad Informática

Seguridad Informática — Unidad 2

MetaGuard

Plataforma web con API RESTful para proteger la privacidad en fotografías y archivos PDF

Alumnos: Antonio de Hoyos Hernández y Luis Emilio Jaras Sánchez

Matrículas: 243776 y 243697

Materia: Seguridad Informática

Unidad: 2 — Propuesta de proyecto (versión corregida)

Fecha: 16 de octubre de 2026

Resumen de la Propuesta

Elemento	Detalle
Objetivos específicos	7
Módulos funcionales	4 (limpieza, contraseñas, amenazas, monitoreo)
Plataforma	Web + API RESTful propia
APIs externas	Have I Been Pwned y VirusTotal
Formatos soportados	JPG, PNG y PDF
Enfoque principal	Privacidad, autenticación y monitoreo

Ámbitos: Privacidad · Autenticación · Cifrado · Monitoreo de amenazas · Pruebas de seguridad

Tabla de contenidos

1. Nombre del Proyecto **3**

2. Planteamiento del Problema a Resolver **4**

2.1. Contexto actual 4

2.2. Justificación 4

2.3. Impacto esperado 5

2.4. Alcance y limitaciones 6

2.5. Soluciones existentes y diferencial 6

3. Modelo de Confianza y Tratamiento de Datos **8**

3.1. Actores y supuestos 8

3.2. Datos que la plataforma conserva 9

3.3. Transparencia y datos personales 9

4. Objetivos **11**

4.1. Objetivo General 11

4.2. Objetivos Específicos 11

5. Funcionalidades del Sistema **14**

6. Stack Tecnológico y Arquitectura **15**

6.1. Controles de seguridad del procesamiento de archivos 17

6.2. Extensiones futuras (fuera del alcance) 17

7. Plan de Trabajo y Riesgos del Proyecto **18**

8. Bibliografía **19**

1. Nombre del Proyecto

MetaGuard: plataforma web con API RESTful propia para detectar, clasificar y eliminar metadatos personales en fotografías y archivos PDF, neutralizar contenido activo peligroso en PDF y proteger las cuentas mediante la verificación de contraseñas filtradas y el monitoreo de eventos de seguridad, procesando los archivos no confiables en un entorno aislado y sin conservarlos más allá del tiempo necesario para descargarlos.

2. Planteamiento del Problema a Resolver

Los archivos que las personas comparten todos los días guardan información invisible que revela más de lo que el usuario cree, pueden incluir contenido activo peligroso, y casi nadie sabe qué datos entrega ni cómo eliminarlos de forma confiable. Además, las cuentas que protegen esa información suelen caer por contraseñas débiles o ya filtradas, sin que el usuario note los intentos de acceso.

2.1 Contexto actual

Casi todos los archivos digitales incluyen metadatos: datos sobre el archivo que no se ven al abrirlo. Una fotografía tomada con un teléfono puede contener las coordenadas GPS exactas, la fecha y la hora, el modelo del dispositivo, números de serie y una **miniatura incrustada** que a veces conserva la imagen original sin recortar. Además del formato EXIF existen bloques XMP, IPTC, perfiles de color y notas del fabricante. Un PDF suele guardar el autor, el programa de creación, las fechas y, en ocasiones, rutas de carpetas del equipo de origen.

Un PDF también puede contener elementos activos: JavaScript embebido, acciones que se ejecutan al abrirlo (`/OpenAction`), acciones asociadas a campos (`/AA`), formularios XFA o archivos adjuntos ocultos. Un archivo limpio de metadatos que conserva estos elementos sigue siendo un riesgo para quien lo recibe.

Al publicar una foto del hogar, enviar un PDF a un cliente o compartir un documento en una red social, la persona puede revelar su ubicación, su identidad o información de su organización sin saberlo. Por último, una plataforma que maneja archivos sensibles es objetivo de ataques contra las cuentas: reutilización de contraseñas filtradas, fuerza bruta y accesos desde ubicaciones desconocidas.

2.2 Justificación

JUS-01 | Es un riesgo de privacidad real y cotidiano

Los metadatos pueden usarse para localizar o identificar a una persona, y el usuario común no tiene forma sencilla de saber que existen.

Resultado esperado: el usuario conoce qué datos lleva cada archivo antes de compartirlo.

JUS-02 | Borrar no basta si la verificación no es independiente

Los datos pueden estar en varios lugares de un mismo archivo (en un PDF: diccionario de información, bloques XMP y anotaciones; en una imagen: EXIF, XMP, IPTC y miniatura). Si la misma biblioteca limpia y verifica, un campo que no reconoce quedaría sin borrar y sin detectar. MetaGuard verifica con herramientas **distintas** a las que usó para limpiar y compara el archivo antes y después.

Resultado esperado: reporte antes/después de cada limpieza, validado por un verificador independiente.

JUS-03 | La herramienta misma es una superficie de ataque

Una plataforma que procesa archivos subidos debe tratarlos como no confiables: limitar su tamaño, aislar el procesamiento y no conservar el contenido. Todo análisis del contenido (incluida la detección del tipo real) ocurre dentro de un contenedor sin red; el contenedor con acceso a red nunca interpreta el archivo. Este diseño es parte central del proyecto.

Resultado esperado: controles de carga y aislamiento verificados con OWASP ZAP y pruebas propias.

JUS-04 | Un archivo sin metadatos puede seguir siendo peligroso

Si solo se eliminan metadatos, el PDF se devuelve con JavaScript, acciones automáticas y adjuntos, y el usuario puede reenviarlo sin saberlo. MetaGuard neutraliza esos elementos y, con consentimiento del usuario, consulta la reputación del archivo por su hash.

Resultado esperado: PDF de prueba con elementos activos detectados y neutralizados en la copia limpia.

JUS-05 | Una contraseña filtrada anula un hash seguro

Aunque las contraseñas se guarden con Argon2id [5], una que ya aparece en listas de atacantes se prueba primero. MetaGuard la rechaza al registrarse o cambiarla, sin enviar la contraseña ni su hash completo, en línea con las recomendaciones del NIST [3].

Resultado esperado: ninguna cuenta nueva se crea con una contraseña conocida como filtrada.

JUS-06 | Sin monitoreo, los ataques pasan inadvertidos; con bloqueo ingenuo, el monitoreo se convierte en un arma

MetaGuard muestra al usuario la actividad de su cuenta, alerta por correo ante comportamiento sospechoso y limita los intentos fallidos sin permitir que un atacante deje fuera a un usuario legítimo.

Resultado esperado: ataques de fuerza bruta y cargas anómalas detectados, visibles y mitigados.

JUS-07 | Cubre varios ejes de la seguridad informática

Privacidad, autenticación fuerte, cifrado, monitoreo de amenazas, seguridad de aplicaciones web y pruebas de penetración, integrados en una sola solución.

Resultado esperado: todos los ejes cubiertos por al menos un objetivo específico.

2.3 Impacto esperado

- **Menor exposición de datos personales:** el usuario identifica qué información sensible lleva un archivo y la elimina antes de compartirlo.
- **Comprensión del riesgo:** cada dato se clasifica como de riesgo alto, medio o bajo y se explica en lenguaje sencillo.

- **Limpieza verificable:** un reporte antes/después, validado de forma independiente, confirma qué se eliminó.
- **Archivos más seguros de compartir:** el contenido activo de los PDF se neutraliza y el reporte indica el resultado de la consulta de reputación.
- **Cuentas más difíciles de vulnerar:** se bloquean contraseñas filtradas y se limita la fuerza bruta sin habilitar el bloqueo malicioso de cuentas ajenas.
- **Visibilidad de la actividad:** el usuario ve los eventos de seguridad de su cuenta y recibe alertas.
- **Minimización de datos desde el diseño:** los archivos no se conservan (máximo 10 minutos para la descarga) y los datos que sí se guardan tienen retención limitada y cifrado (sección 3).
- **Aportación académica:** un caso integrado de privacidad, procesamiento seguro de archivos hostiles, monitoreo y arquitectura cliente-API replicable.

2.4 Alcance y limitaciones

- **Formatos:** JPG, PNG y PDF. Quedan fuera documentos de ofimática, video y procesamiento por lotes.
- **Contenido visible:** eliminar metadatos no elimina lo que se ve (rostros, letreros con direcciones, texto del documento).
- **Marcas ocultas:** no se eliminan marcas dentro de los píxeles ni huellas propias del sensor de la cámara.
- **Recompresión de imágenes:** limpiar una imagen implica re-codificarla; puede variar el peso y la calidad del JPG. El reporte lo indica.
- **Verificación:** comprueba los campos que ExifTool y qpdf saben extraer; no garantiza la ausencia total de información oculta en formatos complejos. Las métricas se expresan sobre un corpus de prueba descrito en el informe, no como garantías generales.
- **Detección de amenazas:** MetaGuard no es un antivirus. Detecta elementos activos conocidos y consulta la reputación por hash; «sin antecedentes» no significa «seguro», y el reporte lo indica.
- **El hash sí sale de la plataforma:** el SHA-256 es un identificador único; VirusTotal puede saber que alguien consultó ese archivo. Por eso la consulta es opcional y requiere consentimiento explícito.
- **Servicios externos:** si Pwned Passwords o VirusTotal no están disponibles, el sistema aplica validación local alternativa y registra el evento, con protección reducida durante la interrupción.
- **Monitoreo:** la detección de IP nueva depende de que Nginx transmita la IP real y de que la API confíe solo en ese proxy. Las direcciones compartidas o cambiantes pueden generar falsas alertas.
- **Operador:** se asume un operador de la plataforma honesto (sección 3); el sistema reduce lo que este puede ver, pero el servidor procesa el archivo en claro durante unos segundos.

2.5 Soluciones existentes y diferencial

Existen herramientas que resuelven partes del problema, pero ninguna integra el flujo completo para un usuario no técnico.

Herramienta	Qué resuelve	Qué no cubre en este caso
ExifTool [9]	Lee y escribe metadatos de muchos formatos.	Orientada a línea de comandos; no explica el riesgo de cada dato, ni neutraliza PDF, ni protege cuentas.
mat2 [10]	Elimina metadatos de varios formatos.	No clasifica el riesgo de cada dato ni ofrece plataforma multiusuario con monitoreo.
Dangerzone [11]	Convierte documentos no confiables en PDF seguros rasterizando las páginas.	El resultado pierde la estructura original; no reporta metadatos por riesgo.
VirusTotal [7]	Reputación y análisis de archivos con múltiples motores.	No elimina metadatos; subir el archivo lo comparte con terceros.

Diferencial de MetaGuard: un flujo unificado que explica el riesgo de cada dato en lenguaje sencillo, limpia, verifica con una herramienta independiente, neutraliza el contenido activo del PDF y protege la cuenta, todo con procesamiento aislado y sin enviar el archivo a terceros.

3. Modelo de Confianza y Tratamiento de Datos

3.1 Actores y supuestos

Declarar quién es confiable y quién no define qué controles son necesarios. Sin este modelo, la promesa de privacidad no se puede evaluar ni probar.

- **Usuario:** legítimo, pero puede cometer errores (reutilizar contraseñas, compartir archivos con datos sensibles).
- **Atacante externo:** sube archivos hostiles, ataca cuentas por fuerza bruta o con contraseñas filtradas, y manipula cabeceras HTTP (por ejemplo, la de IP de origen).
- **Servicios externos (Have I Been Pwned, VirusTotal):** no se les confía información del usuario; solo reciben el prefijo SHA-1 de 5 caracteres y el hash SHA-256, respectivamente.
- **Operador de la plataforma:** se asume honesto, porque es quien despliega el sistema y el servidor debe leer el archivo para procesarlo. Se mitiga reduciendo lo que se conserva: sin archivos en disco persistente, sin contenido en bitácoras y con cifrado de campos sensibles.

3.2 Datos que la plataforma conserva

Dato	Finalidad	Retención	Protección
Archivo subido y copia limpia	Procesar y permitir la descarga	Máximo 10 minutos; se borra al descargar o al expirar	Memoria o carpeta temporal en memoria, AES-256-GCM, token de descarga de un solo uso
Reporte de análisis	Mostrar el antes/después	24 horas o borrado manual por el usuario	Valores sensibles (GPS, serie, autor) cifrados por campo con AES-256-GCM
Eventos de seguridad (incluyen IP)	Monitoreo y alertas	30 días	Cada usuario solo accede a los suyos
Contraseña	Autenticación	Mientras exista la cuenta	Hash Argon2id; nunca en texto claro
Secreto TOTP	MFA	Mientras exista la cuenta	Cifrado AES-256-GCM
Caché de reputación	Respetar el límite de VirusTotal	24 horas	Solo hash y veredicto; sin contenido ni vínculo con el usuario

Una tarea programada purga los datos vencidos y su ejecución se verifica en las pruebas. Si no existiera, la base de datos acumularía coordenadas, autoría e IP de los usuarios y se convertiría en el activo más sensible de la plataforma.

3.3 Transparencia y datos personales

Aunque el proyecto es académico, la plataforma maneja datos personales y se diseña tomando como requisito los principios de la ley mexicana de protección de datos en posesión de particulares [8]:

- **Información:** aviso de privacidad visible antes del registro, que explica qué se conserva, por cuánto tiempo y con qué fin.
- **Consentimiento:** la consulta de reputación a VirusTotal es opcional y se activa por archivo, con la advertencia de que se envía su hash.
- **Proporcionalidad:** solo se guarda lo necesario para el servicio (tabla anterior).
- **Acceso y supresión:** el usuario puede ver sus datos y eliminar su cuenta, lo que borra sus reportes

y eventos.

4. Objetivos

4.1 Objetivo General

Diseñar, desarrollar y evaluar una plataforma web integrada con una API RESTful propia que permita al usuario subir fotografías y archivos PDF, detectar y clasificar por nivel de riesgo los metadatos personales que contienen, eliminarlos y verificar la limpieza con herramientas independientes, neutralizar el contenido activo peligroso de los PDF y proteger las cuentas mediante la verificación de contraseñas filtradas y el monitoreo de eventos de seguridad, procesando los archivos en un entorno aislado y sin conservarlos, con el fin de reducir la exposición involuntaria de datos personales.

4.2 Objetivos Específicos

OBJ-01 | Diseñar la arquitectura y el modelo de amenazas

Definir la arquitectura cliente-API con dos contenedores separados (API con red y contenedor de procesamiento sin red), los límites de confianza, el flujo de datos, el ciclo de vida de los datos (sección 3) y un análisis STRIDE [2] centrado en archivos hostiles, ataques a cuentas y manipulación de cabeceras HTTP. El operador se declara como actor confiable bajo supuestos explícitos.

Medida de éxito: diagramas de arquitectura y de flujo de datos con límites de confianza; matriz STRIDE con al menos 10 amenazas, cada una con su control y la prueba que lo verifica.

OBJ-02 | Desarrollar la API RESTful segura

Implementar y documentar con OpenAPI los endpoints de usuarios, carga, análisis, reportes, descarga y eventos de seguridad, con JWT, MFA por TOTP [4], protección CSRF, autorización por objeto, validación estricta de entradas y descarga mediante un token de un solo uso con vida de 10 minutos. Nginx descarta las cabeceras de IP enviadas por el cliente y la API confía solo en el proxy propio.

Medida de éxito: al menos 12 endpoints; todos exigen autenticación salvo los públicos listados (registro, inicio de sesión y verificación de estado), que tienen límite de tasa; cobertura de pruebas automatizadas de al menos 70 %; una prueba demuestra que una cabecera de IP falsificada no evade el límite de tasa y otra que el archivo ya no existe tras la descarga o la expiración.

OBJ-03 | Implementar el motor de análisis y limpieza

Dentro del contenedor aislado, extraer metadatos de JPG, PNG y PDF y clasificarlos por riesgo (alto, medio, bajo). En imágenes, aplicar primero la orientación y re-codificar los píxeles, lo que descarta EXIF, XMP, IPTC, miniaturas incrustadas, perfil de color y notas del fabricante. En PDF, limpiar el diccionario de información, los bloques XMP y las anotaciones. Verificar con herramientas distintas a las de limpieza (ExifTool y qpdf) y generar el reporte antes/después.

Medida de éxito: en un corpus de 20 archivos con campos de riesgo sembrados y conocidos (GPS, autor, número de serie, miniatura, rutas), 100 % de los campos de riesgo alto eliminados según el verificador independiente; el informe describe la composición del corpus.

OBJ-04 | Construir el frontend y validar la seguridad

Desarrollar el dashboard web (carga, resultados, punto GPS en un mapa autoalojado, descarga, actividad de la cuenta, aviso de privacidad y consentimiento de consulta de reputación) y ejecutar pruebas conforme al OWASP Top 10 [1], con una política CSP estricta que bloquea conexiones a dominios no previstos. El mapa no usa teselas de terceros: de lo contrario, el proveedor del mapa conocería la zona donde se tomó la foto.

Medida de éxito: flujo completo de extremo a extremo; OWASP ZAP, pip-audit, npm audit y Trivy sin hallazgos críticos ni altos abiertos; captura de tráfico del navegador que demuestra que el mapa no hace peticiones a terceros; despliegue con un solo comando.

OBJ-05 | Implementar la verificación de contraseñas filtradas

Al registrar o cambiar contraseña, la API consulta Pwned Passwords [6] mediante k-anonimato: solo se envían los primeros 5 caracteres del hash SHA-1, con el relleno de respuesta (padding) activado para que su tamaño no revele el rango, y la comparación se hace localmente. Si el servicio falla, se aplica una longitud mínima y una lista local de contraseñas comunes, y se registra el evento.

Medida de éxito: 100 % de un conjunto de 20 contraseñas filtradas conocidas rechazadas; captura de tráfico saliente de la API que demuestra que nunca sale la contraseña ni el hash completo; con el servicio simulado como no disponible, el registro sigue funcionando con la validación local.

OBJ-06 | Implementar la detección de contenido peligroso en PDF

Dentro del contenedor aislado, recorrer todos los objetos del PDF (incluidos los comprimidos en flujos de objetos y los de nombres codificados) y eliminar en la copia limpia **todas** las claves de acción y de contenido activo, no solo algunas: `/JavaScript` `/JS` `/OpenAction` `/AA` `/Launch` `/GoToR` `/SubmitForm` `/ImportData` `/RichMedia` `/XFA`, además de los archivos adjuntos (`/EmbeddedFiles` y anotaciones de archivo). Los enlaces `/URI` se listan en el reporte sin consultarlos. Con consentimiento del usuario, la API consulta VirusTotal [7] usando solo el SHA-256 del archivo original **y** de la copia limpia (la que el usuario realmente compartirá), con caché para respetar el límite de consultas.

Medida de éxito: en 10 PDF de prueba con contenido activo (incluidas variantes con nombres codificados y objetos comprimidos), 100 % detectados y neutralizados en la copia limpia, verificado con qpdf; el contenido y los enlaces nunca se envían a terceros; el reporte muestra ambos hashes con su resultado y distingue «sin antecedentes» de «seguro».

OBJ-07 | Implementar el monitoreo y las alertas de seguridad

Registrar siete tipos de evento (inicio de sesión fallido, fallo de MFA, archivo con tipo falso, límite de tasa excedido, acceso desde IP nueva, contenido peligroso detectado, contraseña filtrada rechazada),

mostrarlos en `/security/events` y enviar alerta por correo ante comportamiento sospechoso. Tras 5 fallos consecutivos desde la misma combinación cuenta e IP se aplica un bloqueo temporal progresivo (1, 5 y 15 minutos). Si una cuenta recibe fallos desde muchas IP distintas, **no se bloquea**: se exige un desafío adicional (código TOTP o verificación por correo) y se alerta al dueño. Sin esta distinción, un atacante con varias IP podría dejar fuera a usuarios legítimos.

Medida de éxito: 7 tipos de evento registrados; bloqueo tras 5 fallos del mismo par cuenta e IP verificado; prueba en la que un atacante con otra IP no impide el inicio de sesión del usuario legítimo; alerta por correo recibida en menos de 1 minuto; cada usuario ve únicamente sus propios eventos (pruebas de autorización por objeto).

5. Funcionalidades del Sistema

Módulo	Qué hace	Qué pasa si no existe
Limpieza de metadatos (OBJ-03)	Extrae, clasifica por riesgo, elimina y verifica con herramienta independiente los metadatos de JPG, PNG y PDF.	El usuario comparte archivos con su ubicación, autoría y dispositivo sin saberlo.
Contraseñas filtradas (OBJ-05)	Rechaza contraseñas conocidas en brechas mediante k-anonimato con Pwned Passwords.	Un usuario puede registrarse con una contraseña que los atacantes ya prueban primero, aunque se use Argon2id.
Contenido peligroso (OBJ-06)	Detecta y neutraliza JavaScript, acciones automáticas y adjuntos en PDF; consulta la reputación por hash con consentimiento.	La plataforma devuelve como «limpio» un PDF que sigue siendo peligroso para quien lo reciba.
Monitoreo y alertas (OBJ-07)	Muestra eventos de la cuenta, alerta por correo y aplica bloqueo progresivo o desafío adicional según el patrón de ataque.	Un ataque de fuerza bruta o de cargas maliciosas ocurre sin que nadie lo detecte ni lo detenga; con un bloqueo ingenuo, además, se puede dejar fuera a usuarios legítimos.

6. Stack Tecnológico y Arquitectura

El dashboard consume únicamente los endpoints de la API propia. La separación en dos contenedores es el control central: el que tiene red nunca interpreta archivos, y el que interpreta archivos nunca tiene red. El flujo de una carga es:

1. **Nginx:** termina TLS, limita el tamaño del cuerpo y la tasa de peticiones, y establece la IP real del cliente.
2. **API:** autentica, autoriza, recibe el archivo por flujo con límite de tamaño, calcula el SHA-256 sin interpretar el contenido y lo deja cifrado en memoria temporal.
3. **Contenedor de procesamiento (sin red, sin privilegios):** valida el tipo real y las dimensiones, extrae, clasifica, limpia y verifica con ExifTool y qpdf; devuelve la copia limpia y el reporte.
4. **API:** si el usuario lo autorizó, consulta VirusTotal con los hashes; guarda el reporte con campos sensibles cifrados y emite el token de descarga de un solo uso.
5. **Purga:** el archivo se borra al descargarlo o a los 10 minutos.

Capa	Tecnología	Justificación
Frontend (Web)	React 18, TypeScript, Vite, Tailwind CSS, Leaflet con mapa vectorial autoalojado	Dashboard tipado y rápido; el mapa muestra el punto GPS sin enviar la zona a terceros.
Backend y API	Python 3.12, FastAPI, Pydantic, SQLAlchemy, Alembic	API RESTful con documentación OpenAPI automática y validación estricta de entradas.
Procesamiento (contenedor aislado)	Pillow y piexif (imágenes); pikepdf (PDF); python-magic (tipo real); ExifTool y qpdf (verificadores independientes)	Bibliotecas maduras para limpiar y recorrer la estructura del PDF; la verificación con herramientas distintas evita que un mismo punto ciego pase inadvertido.
Base de datos	PostgreSQL 16	Usuarios, reportes con campos sensibles cifrados, eventos, bitácora de auditoría, contadores de intentos fallidos y caché de reputación con expiración.
Autenticación y autorización	JWT de corta vida en cookies HttpOnly/Secure/SameSite, token anti-CSRF, MFA por TOTP, Argon2id, control de acceso por objeto	Protege las cuentas y evita el acceso a reportes y eventos de otros usuarios (BOLA/IDOR).
Servicios externos	Have I Been Pwned (rango por prefijo SHA-1, con padding); VirusTotal API v3 (hash SHA-256)	Verifican contraseñas y reputación sin enviar la contraseña, el hash completo ni el contenido.
Alertas	SMTP (Mailpit en desarrollo, proveedor SMTP en la demostración)	Notificación ante comportamiento sospechoso sin infraestructura propia compleja.
Cifrado	TLS; AES-256-GCM para datos temporales y campos sensibles	Confidencialidad en tránsito, en procesamiento y en reposo.
Seguridad y calidad	OWASP ZAP, Bandit, pip-audit, npm audit, Trivy, pytest, GitHub Actions	Análisis dinámico, estático y de dependencias, y pruebas automatizadas en cada integración.
Infraestructura	Docker, Docker Compose, Nginx como proxy inverso con límite de tasa	Despliegue reproducible con un solo comando; el procesamiento corre sin privilegios y sin red.

6.1 Controles de seguridad del procesamiento de archivos

- Todo análisis del contenido ocurre en el contenedor sin red, con sistema de archivos de solo lectura, sin privilegios adicionales y con límites de CPU, memoria y tiempo.
- Validación del tipo real del archivo, límite de tamaño y de dimensiones de imagen (contra archivos que se descomprimen a tamaños enormes).
- Procesamiento en memoria o en carpeta temporal en memoria, con borrado inmediato.
- Límite de tasa por usuario y por IP, con IP real controlada por el proxy propio.
- El hash SHA-256 se calcula en la API y es lo único derivado del archivo que se consulta externamente, solo con consentimiento.

6.2 Extensiones futuras (fuera del alcance)

Soporte para documentos de ofimática y video, procesamiento por lotes, detección de datos personales en el texto visible (OCR), rasterización de páginas como modo de alta seguridad para PDF (enfoque de Dangerzone) y extensión de navegador.

7. Plan de Trabajo y Riesgos del Proyecto

Todos los objetivos se mantienen. Para reducir el riesgo de entregar todo a medias, el desarrollo avanza por fases, cada una con un entregable verificable.

Fase	Contenido	Entregable verificable
1. Base	OBJ-01 y OBJ-02: arquitectura, STRIDE, API con autenticación y contenedores separados.	Flujo de carga y descarga con purga funcionando de extremo a extremo.
2. Motor	OBJ-03, OBJ-05 y OBJ-06: limpieza, verificación independiente, contraseñas filtradas y PDF peligroso.	Corpus de prueba con resultados reproducibles.
3. Cierre	OBJ-07 y OBJ-04: monitoreo, dashboard, mapa, MFA y pruebas de seguridad.	Reporte de ZAP y dependencias; despliegue con un solo comando.

Riesgo	Efecto	Mitigación
Alcance amplio para dos integrantes	Entrega incompleta	Fases con entregable; la fase 1 debe funcionar antes de iniciar la 3; revisión semanal de avance.
Límite de consultas de VirusTotal	Reputación no disponible	Caché de 24 horas, consulta opcional y mensaje de «no disponible» en el reporte.
Falsos negativos del analizador	Limpieza incompleta certificada como correcta	Verificador independiente y corpus con campos sembrados conocidos.
Falsas alertas por IP compartida o cambiante	Alertas que el usuario deja de atender	Alertar por IP nueva con confirmación del usuario y umbrales ajustables.

8. Bibliografía

[1] OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

[2] Hernan, S., Lambert, S., Ostwald, T., y Shostack, A. (2006). Uncover security design flaws using the STRIDE approach. *MSDN Magazine*.

[3] Grassi, P. A., et al. (2017). *NIST Special Publication 800-63B: Digital Identity Guidelines, Authentication and Lifecycle Management*. National Institute of Standards and Technology.

[4] M'Raihi, D., Machani, S., Pei, M., y Rydell, J. (2011). *RFC 6238: TOTP, Time-Based One-Time Password Algorithm*. IETF.

[5] Biryukov, A., Dinu, D., Khovratovich, D., y Josefsson, S. (2021). *RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. IETF.

[6] Hunt, T. *Have I Been Pwned: Pwned Passwords API v3*. https://haveibeenpwned.com/API/v3

[7] VirusTotal. *VirusTotal API v3 documentation*. https://docs.virustotal.com/

[8] Ley Federal de Protección de Datos Personales en Posesión de los Particulares. Diario Oficial de la Federación, 20 de marzo de 2025.

[9] Harvey, P. *ExifTool*. https://exiftool.org

[10] *mat2: Metadata Anonymisation Toolkit v2*. https://0xacab.org/jvoisin/mat2

[11] Freedom of the Press Foundation. *Dangerzone*. https://dangerzone.rocks